

Chapitre 3: SQL Dynamique, Collections et Relationnel Objet

2

SQL DYNAMIQUE

3

SQL DYNAMIQUE NATIF

4

SQL Dynamique Natif

Support direct dans le PL/SQL

Exécution de requête dont la structure est inconnue avant l'exécution

Syntaxe

- EXECUTE IMMEDIATE
- OPEN-FOR
- FETCH
- CLOSE

5

Syntaxe

EXECUTE IMMEDIATE

Syntaxe :

```
EXECUTE IMMEDIATE dynamic_string
    [INTO { define_variable
        [, define_variable] ... | record}]
    [USING [IN|OUT|IN OUT] bind_argument
        [, [IN|OUT|IN OUT] bind_argument] ... ];
```

- INTO : requête renvoyant une seule ligne et une colonne à stocker
- USING : arguments

6

SQL dynamique

Ordre LDD

- Exemple Oracle : Création de table

```
CREATE PROCEDURE create_table(
    table_name VARCHAR2, col_specs VARCHAR2) IS
BEGIN
    EXECUTE IMMEDIATE 'CREATE TABLE ' || table_name ||
        ' (' || col_specs || ')';
END;
/
```

- Exemple Oracle : Appel

```
BEGIN
    create_table('EMPLOYEE NAMES',
        'id NUMBER(4) PRIMARY KEY, name VARCHAR2(40)');
END;
/
```

7

SQL dynamique : Exemples Oracle

Suppression de lignes

```
CREATE FUNCTION del_rows(table_name VARCHAR2)
RETURN NUMBER IS
BEGIN
    EXECUTE IMMEDIATE 'DELETE FROM ' || table_name;
    RETURN SQL%ROWCOUNT;
END;

BEGIN DBMS_OUTPUT.PUT_LINE(
    del_rows('EMPLOYEE_NAMES') || ' rows deleted. ');
END;
```

Insertion d'un tuple

```
CREATE PROCEDURE add_row(table_name VARCHAR2,
    id NUMBER, name VARCHAR2) IS
BEGIN
    EXECUTE IMMEDIATE 'INSERT INTO ' || table_name ||
        ' VALUES (:1, :2)' USING id, name;
END;
```

8

Exemple Oracle

Retour d'un tuple

```
CREATE FUNCTION get_emp(emp_id NUMBER)
RETURN employees%ROWTYPE IS
    stmt VARCHAR2(200);
    emprec employees%ROWTYPE;
BEGIN
    stmt := 'SELECT * FROM employees ' ||
        'WHERE employee id = :id';
    EXECUTE IMMEDIATE stmt INTO emprec USING emp_id;
    RETURN emprec;
END;
/

DECLARE
    emprec employees%ROWTYPE := get_emp(100);
BEGIN
    DBMS_OUTPUT.PUT_LINE('Emp: ' || emprec.last_name);
END;
/
```

9

Exemple Oracle

Retour de plusieurs tuples

- OPEN-FOR, FETCH, CLOSE

```
CREATE PROCEDURE list_employees(deptid NUMBER) IS
TYPE emp_refcsr IS REF CURSOR;
emp_cv emp_refcsr;
emprec employees%ROWTYPE;
stmt varchar2(200) := 'SELECT * FROM employees';
BEGIN
IF deptid IS NULL THEN OPEN emp_cv FOR stmt;
ELSE
stmt := stmt || ' WHERE department_id = :id';
OPEN emp_cv FOR stmt USING deptid;
END IF;
LOOP
FETCH emp_cv INTO emprec;
EXIT WHEN emp_cv%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(emprec.department_id ||
'|' || emprec.last_name);
END LOOP;
CLOSE emp_cv;
END;
```

10

Bloc PL/SQL dynamique

Exemple Oracle

```
CREATE FUNCTION annual_sal(emp_id NUMBER)
RETURN NUMBER IS
plsql varchar2(200) :=
'DECLARE ' ||
' emprec employees%ROWTYPE; ' ||
'BEGIN ' ||
' emprec := get_emp(:empid); ' ||
' :res := emprec.salary * 12; ' ||
'END;';
result NUMBER;
BEGIN
EXECUTE IMMEDIATE plsql
USING IN emp_id, OUT result;
RETURN result;
END;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(annual_sal(100))
```

11

Compilation bloc PL/SQL

Avec ALTER

- ALTER PROCEDURE name COMPILE
- ALTER FUNCTION name COMPILE
- ALTER PACKAGE name COMPILE SPECIFICATION
- ALTER PACKAGE name COMPILE BODY

Exemple Oracle

```
CREATE PROCEDURE compile_plsql(name VARCHAR2,
plsql_type VARCHAR2, options VARCHAR2 := NULL) IS
stmt varchar2(200) := 'ALTER ' || plsql_type ||
' ' || name || ' COMPILE';
BEGIN
IF options IS NOT NULL THEN
stmt := stmt || ' ' || options;
END IF;
EXECUTE IMMEDIATE stmt;
END;
/
```

12

DBMS_SQL PACKAGE

13

Exécution

1. OPEN_CURSOR
2. PARSE
3. BIND_VARIABLE or BIND_ARRAY
4. DEFINE_COLUMN, DEFINE_COLUMN_LONG, or DEFINE_ARRAY
5. EXECUTE
6. FETCH_ROWS or EXECUTE_AND_FETCH
7. VARIABLE_VALUE, COLUMN_VALUE, or COLUMN_VALUE_LONG
8. CLOSE_CURSOR

14

Exemple Oracle

Suppression de tuples

```
CREATE OR REPLACE FUNCTION delete_all_rows
(table_name VARCHAR2) RETURN NUMBER IS
  csr_id INTEGER;
  rows_del NUMBER;
BEGIN
  csr_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(csr_id,
    'DELETE FROM ' || table_name, DBMS_SQL.NATIVE);
  rows_del := DBMS_SQL.EXECUTE(csr_id);
  DBMS_SQL.CLOSE_CURSOR(csr_id);
  RETURN rows_del;
END;
/

CREATE table temp_emp as select * from employees;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Rows Deleted: ' ||
    delete_all_rows('temp_emp'));
END;
/
```

16

Exemple Oracle

Ordre LMD avec paramètres

```
CREATE PROCEDURE insert_row (table_name VARCHAR2,
id VARCHAR2, name VARCHAR2, region NUMBER) IS
  csr_id INTEGER;
  stmt VARCHAR2(200);
  rows_added NUMBER;
BEGIN
  stmt := 'INSERT INTO ' || table_name ||
    ' VALUES (:cid, :cname, :rid)';
  csr_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(csr_id, stmt, DBMS_SQL.NATIVE);
  DBMS_SQL.BIND_VARIABLE(csr_id, ':cid', id);
  DBMS_SQL.BIND_VARIABLE(csr_id, ':cname', name);
  DBMS_SQL.BIND_VARIABLE(csr_id, ':rid', region);
  rows_added := DBMS_SQL.EXECUTE(csr_id);
  DBMS_SQL.CLOSE_CURSOR(csr_id);
  DBMS_OUTPUT.PUT_LINE(rows_added || ' row added');
END;
/
```

17

SQL dynamique versus DBMS_SQL

SQL dynamique

- Plus facile à utiliser
- Nécessite moins de code
- Supporte tous les types PL/SQL y compris ceux utilisateur
- Stocke directement des tuples dans des variables PL/SQL

18

COLLECTIONS ET RELATIONNEL OBJET

29

Collections Oracle

Un ensemble d'objets tous du même type

Fonctionnent comme les tableaux

Peuvent contenir des instances d'un type d'objet particulier ou être « attributs » d'un type d'objet

30

Différents types

Les tableaux associatifs

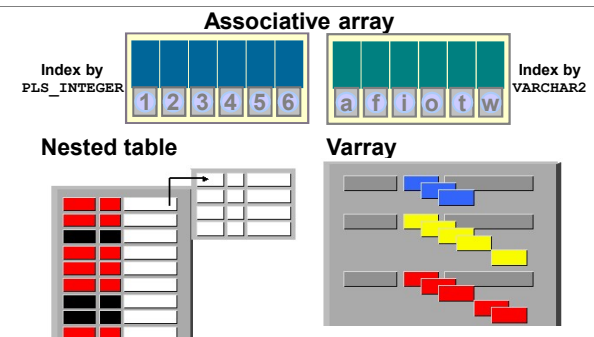
- Indexés par des chaînes de caractères
- INDEX BY pls_integer ou BINARY_INTEGER

Nested tables / Tables imbriquées

Varrays

31

Collections (doc Oracle.com)



32

Tableaux associatifs

Indexés par des chaînes de caractères

Structures mémoire plus rapides que des tables classiques

33

Exemple Oracle

En PL/SQL (string-indexed):

```
TYPE type_name IS TABLE OF element_type INDEX BY VARCHAR2(size)

CREATE OR REPLACE PROCEDURE report_credit
(p_last_name customers.cust_last_name%TYPE, p_credit_limit customers.credit_limit%TYPE)
IS
  TYPE typ_name IS TABLE OF customers%ROWTYPE
    INDEX BY customers.cust_email%TYPE;
  v_by_cust_email typ_name;
  i VARCHAR2(30);

  PROCEDURE load_arrays IS
  BEGIN
    FOR rec IN (SELECT * FROM customers WHERE cust_email IS NOT NULL)
    LOOP
      -- Load up the array in single pass to database table.
      v_by_cust_email(rec.cust_email) := rec;
    END LOOP;
  END;

  ...
```

34

Exemple Oracle : peupler le tableau

```
...
BEGIN
  load_arrays;
  i:= v_by_cust_email.FIRST;
  dbms_output.put_line ('For credit amount of: ' || p_credit_limit);
  WHILE i IS NOT NULL LOOP
    IF v_by_cust_email(i).cust_last_name = p_last_name
    AND v_by_cust_email(i).credit_limit > p_credit_limit
    THEN dbms_output.put_line ('Customer'||
      v_by_cust_email(i).cust_last_name || ': ' ||
      v_by_cust_email(i).cust_email || ' has credit limit of: ' ||
      v_by_cust_email(i).credit_limit);
    END IF;
    i := v_by_cust_email.NEXT(i);
  END LOOP;
END report_credit;
/
```

35

Tables imbriquées

Une table dans une table

Taille non limitée a priori

En SQL et en PL/SQL

Accès direct à des lignes particulières comme dans un tableau

37

Table de stockage

Exemple Oracle

ORDID	SUPPLIER	REQUESTER	ORDERED	ITEMS
500	50	5000	30-OCT-07	
800	80	8000	31-OCT-07	

pOrder nested table

NESTED_TABLE_ID	PRODID	PRICE
55	555	
56	566	
57	577	
NESTED_TABLE_ID	PRODID	PRICE
88	888	

38

Création

Syntaxe SQL

```
CREATE [OR REPLACE] TYPE type_name AS TABLE OF  
element_datatype [NOT NULL];
```

Syntaxe PL/SQL:

```
TYPE type_name IS TABLE OF element_datatype  
[NOT NULL];
```

39

Syntaxe

1. Déclaration du type OBJECT
2. Déclaration de la table imbriquée comme colonne

40

Exemple Oracle

```
CREATE TYPE typ_item AS OBJECT --create object  
(prodid NUMBER(5),  
 price NUMBER(7,2) )  
/  
  
CREATE TYPE typ_item_nst -- define nested table type  
AS TABLE OF typ_item  
/  
  
CREATE TABLE pOrder ( -- create database table  
 ordid NUMBER(5),  
 supplier NUMBER(5),  
 requester NUMBER(4),  
 orderedDATE,  
 items typ_item_nst)  
 NESTED TABLE items STORE AS item_stor_tab  
/
```

41

Peuplement : Exemple Oracle

```
INSERT INTO pOrder
VALUES (500, 50, 5000, sysdate, typ_item_nst(
    typ_item(55, 555),
    typ_item(56, 566),
    typ_item(57, 577)));
```

```
INSERT INTO pOrder
VALUES (800, 80, 8000, sysdate, typ_item_nst (typ_item (88, 888)));
```

Table imbriquée pOrder

ORDID	SUPPLIER	REQUESTER	ORDERED	ITEMS		PRODID	PRICE
500	50	5000	30-OCT-07		→	55	555
						56	566
						57	577
800	80	8000	31-OCT-07		→	88	888

42

Utilisation

Requête classique

```
SELECT * FROM porder;
```

```

ORDID  SUPPLIER  REQUESTER  ORDERED
-----
ITEMS (PRODID, PRICE)
-----
      500      50      5000 31-OCT-07
TYP_ITEM NST(TYP_ITEM(55, 555), TYP_ITEM(56, 566), TYP_ITEM(57, 577))
      800      80      8000 31-OCT-07
TYP_ITEM_NST (TYP_ITEM(88, 888))
```

43

Utilisation

Requête avec la fonction TABLE

```
SELECT p2.ordid, p1.*
FROM porder p2, TABLE(p2.items) p1;
```

ORDID	PRODID	PRICE
800	88	888
500	57	577
500	55	555
500	56	566

44

Accès aux éléments

Via leur indice

◦ Syntaxe `collection_name(subscript)`

◦ Exemple

```
v_with_discount(i)
```

Accès aux champs : exemple

```
p_new_items(i).prodid
```

45

Exemple Oracle PL/SQL

```
CREATE OR REPLACE PROCEDURE add_order_items
(p_ordid NUMBER, p_new_items typ_item_nst)
IS
  v_num_items      NUMBER;
  v_with_discount  typ_item_nst;
BEGIN
  v_num_items := p_new_items.COUNT;
  v_with_discount := p_new_items;
  IF v_num_items > 2 THEN
    --ordering more than 2 items gives a 5% discount
    FOR i IN 1..v_num_items LOOP
      v_with_discount(i) :=
        typ_item(p_new_items(i).prodid,
          p_new_items(i).price*.95);
    END LOOP;
  END IF;
  UPDATE pOrder
  SET items = v_with_discount
  WHERE ordid = p_ordid;
END;
```

46

Utilisation en PL/SQL

```
-- caller pgm:
DECLARE
  v_form_items typ_item_nst:= typ_item_nst();
BEGIN
  -- let's say the form holds 4 items
  v_form_items.EXTEND(4);
  v_form_items(1) := typ_item(1804, 65);
  v_form_items(2) := typ_item(3172, 42);
  v_form_items(3) := typ_item(3337, 800);
  v_form_items(4) := typ_item(2144, 14);
  add_order_items(800, v_form_items);
END;
```

Données de la table imbriquée

v_form_items	
PROIDID	PRICE
1804	65
3172	42
3337	800
2144	14

ORDID	SUPPLIER	REQUESTER	ORDERED	ITEMS
500	50	5000	30-OCT-07	
800	80	8000	31-OCT-07	

PROIDID	PRICE
1804	65
3172	42
3337	800
2144	14

47

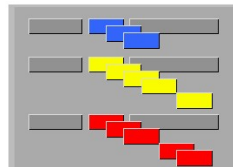
VARRAYS

En SQL

```
CREATE [OR REPLACE] TYPE type_name AS VARRAY(max_elements) OF
  element_datatype [NOT NULL];
```

En PL/SQL

```
TYPE type_name IS VARRAY (max_elements) OF element_datatype [NOT
NULL];
```



48

Mise en oeuvre

1. Définition du type d'objet
2. Déclaration d'une collection de ce type
3. Utilisation SQL ou PL/SQL

49

Exemple Oracle

```
CREATE TYPE typ_Project AS OBJECT( --create object
  project_no NUMBER(4),
  title      VARCHAR2(35),
  cost       NUMBER(12,2)
)

CREATE TYPE typ_ProjectList AS VARRAY (50) OF typ_Project
/

CREATE TABLE department ( -- create database table
  dept_id NUMBER(2),
  name     VARCHAR2(25),
  budget   NUMBER(12,2),
  projects typ_ProjectList) -- declare varray as column
/
```

50

Utilisation : Exemple Oracle

```
INSERT INTO department
VALUES (10, 'Executive Administration', 30000000,
  typ_ProjectList(
    typ_Project(1001, 'Travel Monitor', 400000),
    typ_Project(1002, 'Open World', 10000000)));

INSERT INTO department
VALUES (20, 'Information Technology', 5000000,
  typ_ProjectList(
    typ_Project(2001, 'DB11gR2', 900000)));
```

Table DEPARTMENT					
DEPT_ID	NAME	BUDGET	PROJECTS		
			PROJECT_NO	TITLE	COSTS
10	Executive Administration	30000000	1001.	Travel Monitor	400000
			1002.	Open World	10000000
20	Information Technology	5000000	2001	DB11gR2	900000

51

Requêtes

```
SELECT * FROM department;
```

```

DEPT_ID NAME                      BUDGET
-----
PROJECTS(PROJECT_NO, TITLE, COST)
-----
10 Executive Administration      30000000
TYP_PROJECTLIST(TYP_PROJECT(1001, 'Travel Monitor', 400000),
TYP_PROJECT(1002, 'Open World', 10000000))
20 Information Technology        5000000
TYP_PROJECTLIST(TYP_PROJECT(2001, 'DB11gR2', 900000))
```

52

Requêtes

```
SELECT d2.dept_id, d2.name, d1.*
FROM department d2, TABLE(d2.projects) d1;
```

```

DEPT_ID NAME                      PROJECT_NO TITLE          COST
-----
10 Executive Administration      1001 Travel Monitor    400000
10 Executive Administration      1002 Open World      10000000
20 Information Technology        2001 DB11gR2         900000
```

53

Collections en PL/SQL

Utilisation possible dans les procédures et les fonctions comme paramètres

Utilisation possible comme type de retour de fonction via le RETURN

54

Exemple Oracle

```
CREATE OR REPLACE PACKAGE manage_dept_proj
AS
  PROCEDURE allocate_new_proj_list
    (p_dept_id NUMBER, p_name VARCHAR2, p_budget NUMBER);
  FUNCTION get_dept_project (p_dept_id NUMBER)
    RETURN typ_projectlist;
  PROCEDURE update_a_project
    (p_deptno NUMBER, p_new_project typ_Project,
     p_position NUMBER);
  FUNCTION manipulate_project (p_dept_id NUMBER)
    RETURN typ_projectlist;
  FUNCTION check_costs (p_project_list typ_projectlist)
    RETURN boolean;
END manage_dept_proj;
```

55

Initialisation

3 possibilités

- Utilisation d'un constructeur
- A partir de la BD
- En assignant directement une autre variable de ce type collection

58

Initialisation : Exemple Oracle

```
PROCEDURE allocate_new_proj_list
  (p_dept_id NUMBER, p_name VARCHAR2, p_budget NUMBER)
IS
  v_accounting_project typ_projectlist;
BEGIN
  -- this example uses a constructor
  v_accounting_project :=
    typ_ProjectList
      (typ_Project (1, 'Dsgn New Expense Rpt', 3250),
       typ_Project (2, 'Outsource Payroll', 12350),
       typ_Project (3, 'Audit Accounts Payable', 1425));
  INSERT INTO department
    VALUES(p_dept_id, p_name, p_budget, v_accounting_project);
END allocate_new_proj_list;
```

59

Exemple Oracle

```
FUNCTION get_dept_project (p_dept_id NUMBER)
RETURN typ_projectlist
IS
v_accounting_project typ_projectlist;
BEGIN -- this example uses a fetch from the database
SELECT projects INTO v_accounting_project
FROM department WHERE dept_id = p_dept_id;
RETURN v_accounting_project;
END get_dept_project;
```

```
FUNCTION manipulate_project (p_dept_id NUMBER)
RETURN typ_projectlist
IS
v_accounting_project typ_projectlist;
v_changed_list typ_projectlist;
BEGIN
SELECT projects INTO v_accounting_project FROM department
WHERE dept_id = p_dept_id;
-- this example assigns one collection to another
v_changed_list := v_accounting_project;
RETURN v_changed_list;
END manipulate_project;
```

60

Exemple Oracle

```
-- sample caller program to the manipulate_project function
DECLARE
v_result_list typ_projectlist;
BEGIN
v_result_list := manage_dept_proj.manipulate_project(10);
FOR i IN 1..v_result_list.COUNT LOOP
dbms_output.put_line('Project #: '
||v_result_list(i).project_no);
dbms_output.put_line('Title: '||v_result_list(i).title);
dbms_output.put_line('Cost: ' ||v_result_list(i).cost);
END LOOP;
END;
```

```
Project #: 1001
Title: Travel Monitor
Cost: 400000
Project #: 1002
Title: Open World
Cost: 10000000
```

61

Méthodes

EXISTS
COUNT
LIMIT
FIRST & LAST
PRIOR & NEXT
EXTEND
TRIM
DELETE

```
collection_name.method_name [(parameters)]
```

62

Utilisation : Exemple Oracle

```
FUNCTION check_costs (p_project_list typ_projectlist)
RETURN boolean
IS
c_max_allowed NUMBER := 10000000;
i INTEGER;
v_flag BOOLEAN := FALSE;
BEGIN
i := p_project_list.FIRST ;
WHILE i IS NOT NULL LOOP
IF p_project_list(i).cost > c_max_allowed then
v_flag := TRUE;
dbms_output.put_line (p_project_list(i).title || '
exceeded allowable budget. ');
RETURN TRUE;
END IF;
i := p_project_list.NEXT(i);
END LOOP;
RETURN null;
END check_costs;
```

64

Exemple : Exemple Oracle

```
-- sample caller program to check_costs
set serveroutput on
DECLARE
  v_project_list typ_projectlist;
BEGIN
  v_project_list := typ_ProjectList(
    typ_Project (1,'Dsgn New Expense Rpt', 3250),
    typ_Project (2, 'Outsource Payroll', 120000),
    typ_Project (3, 'Audit Accounts Payable',14250000));
  IF manage_dept_proj.check_costs(v_project_list) THEN
    dbms_output.put_line('Project rejected: overbudget');
  ELSE
    dbms_output.put_line('Project accepted, fill out forms.');
```

V_PROJECT_LIST

PROJECT_NO	TITLE	COSTS
1	Dsgn New Expense Rpt	3250
2	Outsource Payroll	120000
3	Audit Accounts Payable	14250000

65

Manipuler les éléments : Exemple Oracle

```
PROCEDURE update_a_project
(p_deptno NUMBER, p_new_project typ_Project, p_position NUMBER)
IS
  v_my_projects typ_ProjectList;
BEGIN
  v_my_projects := get_dept_project (p_deptno);
  v_my_projects.EXTEND; --make room for new project
  /* Move varray elements forward */
  FOR i IN REVERSE p_position..v_my_projects.LAST - 1 LOOP
    v_my_projects(i + 1) := v_my_projects(i);
  END LOOP;
  v_my_projects(p_position) := p_new_project; -- insert new one
  UPDATE department SET projects = v_my_projects WHERE dept_id = p_deptno;
END update_a_project;
```

66

Exemple Oracle

```
-- check the table prior to the update:
SELECT d2.dept_id, d2.name, d1.*
FROM department d2, TABLE(d2.projects) d1;
```

DEPT_ID	NAME	PROJECT_NO	TITLE	COST
10	Executive Administration	1001	Travel Monitor	400000
10	Executive Administration	1002	Open World	10000000
20	Information Technology	2001	DB11gR2	900000

```
-- caller program to update_a_project
BEGIN
  manage_dept_proj.update_a_project(20,
    typ_Project(2002, 'AQM', 80000), 2);
END;
```

```
-- check the table after the update:
SELECT d2.dept_id, d2.name, d1.*
FROM department d2, TABLE(d2.projects) d1;
```

DEPT_ID	NAME	PROJECT_NO	TITLE	COST
10	Executive Administration	1001	Travel Monitor	400000
10	Executive Administration	1002	Open World	10000000
20	Information Technology	2001	DB11gR2	900000
20	Information Technology	2002	AQM	80000

67

Gestion des Exceptions

Exceptions usuelles avec les collections

- COLLECTION_IS_NULL
- NO_DATA_FOUND
- SUBSCRIPT_BEYOND_COUNT
- SUBSCRIPT_OUTSIDE_LIMIT
- VALUE_ERROR

68

Exemple Oracle

```
DECLARE
  TYPE NumList IS TABLE OF NUMBER;
  nums NumList;          -- atomically null
BEGIN
  /* Assume execution continues despite the raised exceptions.
  */
  nums(1) := 1;           -- raises COLLECTION_IS_NULL
  nums := NumList(1,2);   -- initialize table
  nums(NULL) := 3         -- raises VALUE_ERROR
  nums(0) := 3;           -- raises SUBSCRIPT_OUTSIDE_LIMIT
  nums(3) := 3;           -- raises SUBSCRIPT_BEYOND_COUNT
  nums.DELETE(1);         -- delete element 1
  IF nums(1) = 1 THEN     -- raises NO_DATA_FOUND
  ...
```

69

Utilisation des collections

Varrays :

- Moins d'accès disque, + efficace
- Ordre des éléments préservé

71

Utilisation des collections

Tables imbriquées : pour de grandes quantités de données

Après suppression libérer la mémoire avec DBMS_SESSION.FREE_UNUSED_USER_MEMORY

72

Exercices

Pour chaque employé, afficher :

son nom,

son intitulé de poste,

son salaire, nom du département

le salaire moyen de son département,

la différence entre son salaire et celui de son manager,

son rang de salaire dans son département,

Un indice de performance basée sur son salaire (s'il est supérieur ou = à 1.2 la moyenne des salaires de son département → Top, s'il est <0,8 sous indice sinon Dans la moyenne

73

Exercices

Créer une collection de salaires, la remplir avec les salaires des employés d'un département donné, puis afficher les employés dont le salaire est **supérieur à la moyenne** de cette collection.

Utiliser TABLE() pour interroger la collection comme une table.

Exercices

Créer une structure de données qui :

regroupe les **départements**,

contient pour chaque département une **collection d'employés**,

et pour chaque employé, une **collection d'historiques de salaires** (simulée).